

Operating Systems and System Programming

Practical – 6

Aim

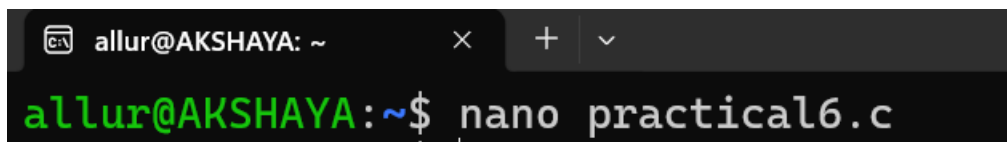
To implement client-server communication using named pipes (FIFOs), where clients send messages and the server processes and responds to them, and to develop a POSIX signal handling program to capture SIGINT, SIGTERM, and SIGUSR1 and demonstrate asynchronous event handling.

Algorithm

1. Start the program.
2. Create named pipes using mkfifo() for client-server communication.
3. Open the pipes and allow the client to send messages to the server.
4. The server reads, processes, and responds to client messages.
5. Test communication with multiple clients and observe FIFO behavior.
6. Register signal handlers for SIGINT, SIGTERM, and SIGUSR1.
7. Display the process ID and wait for incoming signals.
8. When a signal is received, execute the corresponding signal handler.
9. Display the signal received and terminate when appropriate.
10. Stop the program.

Task-1:

Step-1:

A terminal window with a dark background. The prompt is 'allur@AKSHAYA: ~'. The command 'nano practical6.c' is being entered. The terminal has a title bar with a window icon, the text 'allur@AKSHAYA: ~', and window control buttons (close, maximize, and a dropdown arrow).

Step-2: server.c

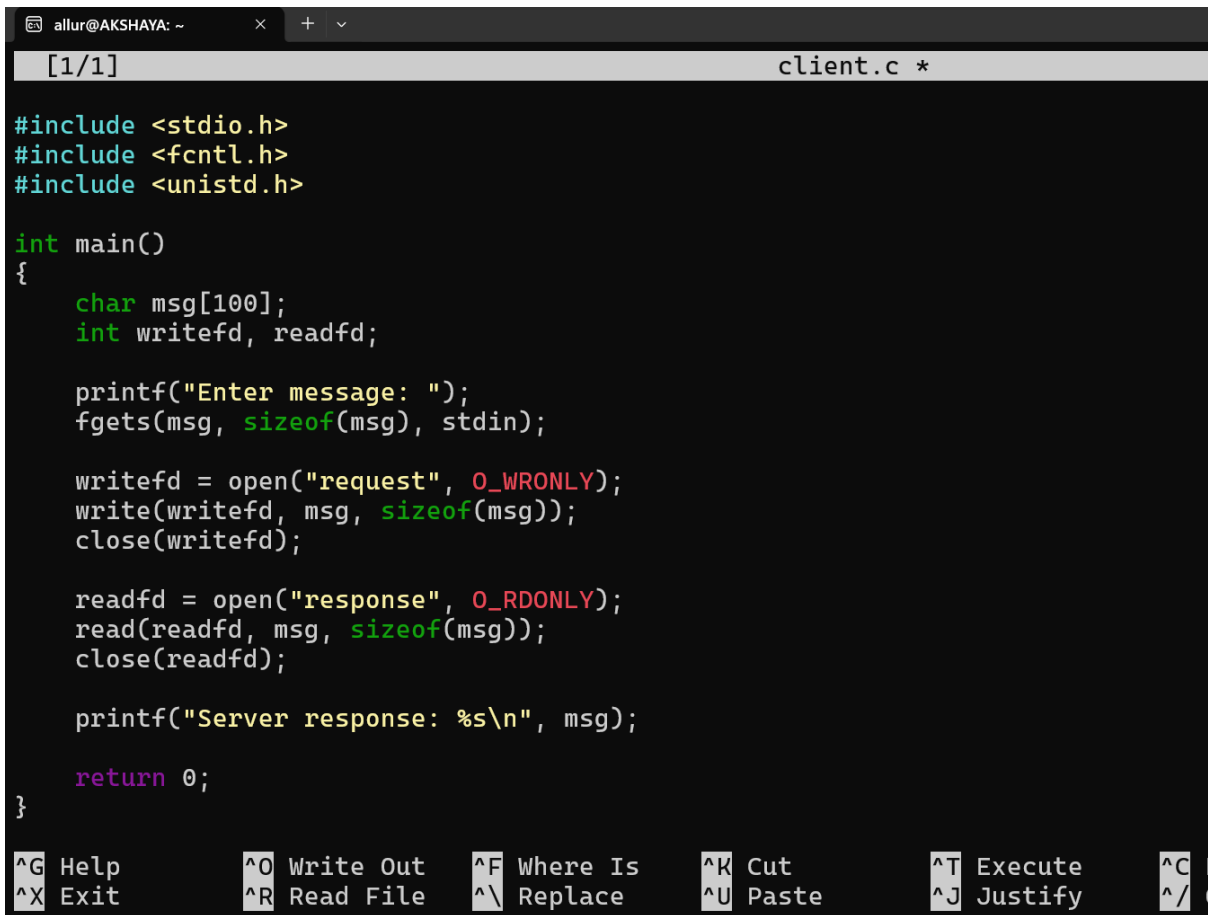
```
allur@AKSHAYA: ~  
GNU nano 8.7.1 practical6.c *  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <fcntl.h>  
#include <sys/stat.h>  
  
int main()  
{  
    char msg[100];  
    int readfd, writefd;  
  
    mkfifo("request", 0666);  
    mkfifo("response", 0666);  
  
    printf("Server waiting for client...\n");  
  
    while (1)  
    {  
        readfd = open("request", O_RDONLY);  
        read(readfd, msg, sizeof(msg));  
        close(readfd);  
  
        printf("Client message: %s\n", msg);  
  
        writefd = open("response", O_WRONLY);  
        write(writefd, "Message received", 17);  
  
        printf("Server message: %s\n", msg);  
  
        writefd = open("response", O_WRONLY);  
        write(writefd, "Message received", 17);  
        close(writefd);  
    }  
  
    return 0;  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Lo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go

```
        writefd = open("response", O_WRONLY);  
        write(writefd, "Message received", 17);  
        close(writefd);  
    }  
  
    return 0;  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Step-3: client.c

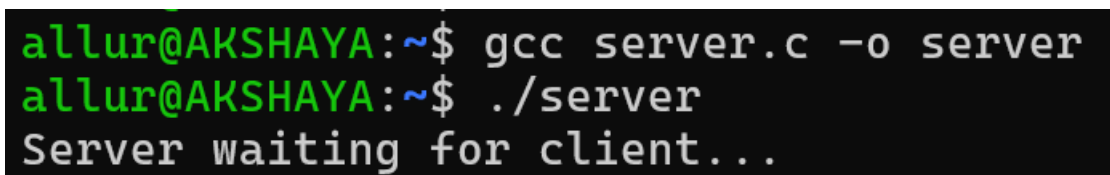


```
[1/1] client.c *  
  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
int main()  
{  
    char msg[100];  
    int writefd, readfd;  
  
    printf("Enter message: ");  
    fgets(msg, sizeof(msg), stdin);  
  
    writefd = open("request", O_WRONLY);  
    write(writefd, msg, sizeof(msg));  
    close(writefd);  
  
    readfd = open("response", O_RDONLY);  
    read(readfd, msg, sizeof(msg));  
    close(readfd);  
  
    printf("Server response: %s\n", msg);  
  
    return 0;  
}
```

Help Write Out Where Is Cut Execute
Exit Read File Replace Paste Justify

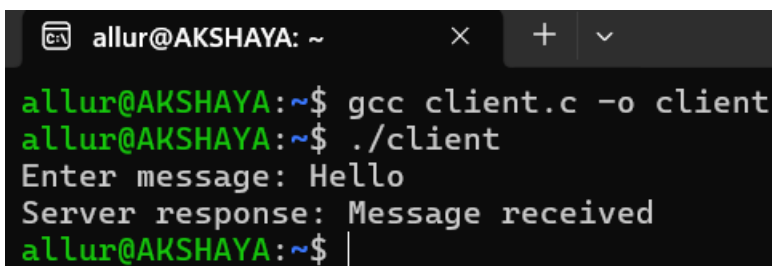
Step-4:

Open Terminal 1 for the server:



```
allur@AKSHAYA:~$ gcc server.c -o server  
allur@AKSHAYA:~$ ./server  
Server waiting for client...
```

Open Terminal 2 for the client:



```
allur@AKSHAYA:~$ gcc client.c -o client  
allur@AKSHAYA:~$ ./client  
Enter message: Hello  
Server response: Message received  
allur@AKSHAYA:~$ |
```

Then in Terminal 1 you get

```
allur@AKSHAYA:~$ gcc server.c -o server
allur@AKSHAYA:~$ ./server
Server waiting for client...
Client message: Hello
```

Task-2:

Step-1:

```
allur@AKSHAYA: ~
allur@AKSHAYA:~$ nano signal.c
```

Step-2:

```
GNU nano 8.7.1 signal.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
void handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received\n");
    else if (sig == SIGTERM)
        printf("\nSIGTERM received\n");
    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received\n");

    if (sig == SIGINT || sig == SIGTERM)
        exit(0);
}
int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
        pause();

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Step-3:

```
allur@AKSHAYA:~$ gcc signal.c -o signal
allur@AKSHAYA:~$ ./signal
Process ID: 749
Waiting for signals...
```

Result

Client-server communication was successfully implemented using named pipes, and FIFO behavior with multiple clients was observed. A POSIX signal handling program was also implemented to capture SIGINT, SIGTERM, and SIGUSR1, demonstrating asynchronous event handling and the role of signal handlers.